Linux网络--应用层DNS、代理与内网穿透

- Linux网络--应用层DNS、代理与内网穿透
- 1. DNS：把域名解析成地址
  - 1.1 为什么需要域名系统
  - 1.2 hosts 文件仍然存在
  - 1.3 DNS 的层级结构
  - 1.4 递归查询与迭代查询
  - 1.5 缓存与 TTL
  - 1.6 一个域名不一定只有一个 IP
  - 1.7 CNAME
  - 1.8 常见记录类型
  - 1.9 使用 dig 分析
  - 1.10 常见 DNS 状态
  - 1.11 DNS 使用的传输协议
  - 1.12 输入 URL 后的实际路径
- 2. 代理服务器
  - 2.1 代理的共同结构
  - 2.2 正向代理代表客户端
  - 2.3 正向代理的用途
  - 2.4 HTTP 代理
  - 2.5 HTTPS CONNECT 隧道
  - 2.6 反向代理代表服务器
  - 2.7 CDN 与反向代理
  - 2.8 代理不全是第七层
- 3. NAT 与代理的区别
- 4. 内网穿透
  - 4.1 NAT 为什么阻挡入站
  - 4.2 NAT 打洞
  - 4.3 打洞与中继
- 5. frp 反向隧道
  - 5.1 frps 与 frpc
  - 5.2 为什么 frp 不是 NAT 打洞
  - 5.3 服务端配置
  - 5.4 内网客户端配置
  - 5.5 暴露 Nginx 服务
  - 5.6 启动与日志

- 6. DNS、代理与内网穿透如何串联
  - 6.1 为什么必须分清各层责任
- 7. 安全边界
  - 7.1 DNS 安全
  - 7.2 代理安全
  - 7.3 内网穿透安全
- 8. 系列总回顾
- 9. 本篇小结

Linux网络--应用层DNS、代理与内网穿透

> **系列导航**
>
> 上一篇：Linux网络--网络层MTU与IP分片
> 本篇：DNS、代理与内网穿透
> 系列终点：从域名访问到跨网络服务交付

前面的文章已经从 IP 地址讲到了路由、链路层、ARP 和分片，但真实用户很少直接输入 IP。浏览器首先需要把域名解析成地址；企业和服务平台还经常通过代理隐藏或保护通信端点；当服务藏在 NAT 后方时，又需要反向隧道才能让公网访问进来。

本篇是系列最后一篇，解决三个与应用入口直接相关的问题：

1. 域名怎样变成 IP；
2. 代理服务器代表谁访问谁；
3. 公网无法直接进入内网时，反向隧道怎样建立通路。

1. DNS：把域名解析成地址

1.1 为什么需要域名系统

IP 地址适合路由，却不适合人记忆。早期互联网依赖一个集中维护的 `hosts` 文件：

```
IP 地址      主机名
```

当新设备加入或地址发生变化时，需要修改中央文件，其他计算机还要同步更新。网络规模扩大后，集中管理很快变得不可维护。

DNS 把命名空间分成层次，让不同组织维护自己的区域：

```
根
└── com
    └── example
        └── www
```

对应域名：

```
www.example.com.
```

末尾的点代表根。日常书写通常省略它，所以写成 `www.example.com`。

1.2 hosts 文件仍然存在

Linux 常见位置：

```
/etc/hosts
```

Windows 常见位置：

```
C:\Windows\System32\drivers\etc\hosts
```

`hosts` 适合少量静态映射，例如开发环境：

```
127.0.0.1 api.local
```

但 `hosts` 是否优先于 DNS 取决于系统名称服务配置。Linux 的 `/etc/nsswitch.conf` 可以决定 `files`、`dns`、`systemd-resolved` 等来源的查询顺序，不能概括为“所有系统永远先查 hosts”。

1.3 DNS 的层级结构

以 `www.example.com` 为例：

层级	名称	作用
根	`.`	指向各顶级域
顶级域	`.com`、`.org`、`.cn`	管理下层域名空间
注册域名	`example.com`	由组织或企业注册和维护
子域名	`www.example.com`	表示具体服务或主机

`www` 不是协议名称，只是网站主机的常见命名习惯。现代服务也可能使用：

```
api.example.com
static.example.com
www.example.com
```

它们可以指向不同服务器或同一反向代理。

1.4 递归查询与迭代查询

浏览器一般不会直接向根服务器逐级询问，而是把域名交给系统解析器，再由递归解析器完成查询。

```
浏览器
-> 操作系统 stub resolver
-> 本地/运营商递归解析器
-> 根服务器
-> .com 顶级域服务器
-> example.com 权威服务器
```

客户端通常向递归解析器提出：

> 请给我 `www.example.com` 的最终地址。

递归解析器则可能逐级询问根、顶级域和权威服务器。这种由递归解析器代问的分步过程称为迭代查询。

根服务器通常只返回“去问 `.com` 服务器”的推荐信息，不直接保存所有网站最终 IP。

1.5 缓存与 TTL

DNS 的规模能够支撑全球访问，缓存非常关键。

```
首次查询：递归解析器访问权威服务器
后续查询：解析器命中缓存
TTL 到期：重新查询
```

缓存可能存在于：

- 浏览器；
- 操作系统；
- systemd-resolved 或本地 DNS 服务；
- 运营商递归解析器；
- 公共 DNS 解析器；
- CDN 调度层。

TTL 越短，更新越快，但权威服务器压力越大；TTL 越长，缓存效率越高，但地址变更传播越慢。

1.6 一个域名不一定只有一个 IP

一个域名可以返回多个 A 或 AAAA 记录：

```
www.example.com A 203.0.113.10
www.example.com A 203.0.113.11
```

因此：

- 不同用户可能得到不同 IP；
- 同一用户在 TTL 内通常复用缓存结果；
- 解析结果可能受地域、运营商、健康检查和负载影响；
- 服务可以随时更换地址，应用不应永久保存本次解析结果。

IPv4 地址记录为 A，IPv6 地址记录为 AAAA。

1.7 CNAME

课堂中的 `dig` 结果可能先出现：

```
www.baidu.com -> www.a.shifen.com
```

这是 CNAME：

```
一个域名是另一个域名的别名
```

解析器继续查询别名，直到得到 A/AAAA 记录。

CNAME 适合把业务域名统一指向 CDN 或托管服务，但根域名处使用 CNAME 有限制，因为根域名通常还必须有 SOA、NS 等记录。

1.8 常见记录类型

类型	用途
A	IPv4 地址
AAAA	IPv6 地址
CNAME	域名别名
MX	邮件交换
NS	区域权威服务器
TXT	策略、验证和说明信息
SOA	区域管理信息

1.9 使用 dig 分析

安装：

```
sudo dnf install bind-utils
```

或：

```
sudo apt install dnsutils
```

基础查询：

```
dig www.example.com
```

输出示例：

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR
;; QUESTION SECTION:
;www.example.com. IN A

;; ANSWER SECTION:
www.example.com. 300 IN CNAME www.example.cdn.example.
www.example.cdn.example. 60 IN A 203.0.113.10
```

各部分含义：

字段	含义
`status: NOERROR`	查询成功
QUESTION	查询的域名和记录类型
ANSWER	CNAME、A 或 AAAA 等结果
`300`、`60`	当前结果剩余 TTL
`SERVER`	实际响应的递归解析器
`Query time`	解析器返回结果的耗时

指定记录类型：

```
dig example.com A
dig example.com AAAA
dig example.com MX
dig example.com NS
```

指定解析服务器：

```
dig @1.1.1.1 example.com
```

1.10 常见 DNS 状态

状态	含义
NOERROR	查询完成；答案可能为空
NXDOMAIN	域名不存在
NODATA	域名存在，但没有所查询记录类型
SERVFAIL	解析器或权威服务器失败，或 DNSSEC 验证失败
REFUSED	服务器拒绝查询

“网页打不开”不能直接等同于 DNS 失败。可以先做：

```
dig example.com
getent hosts example.com
```

再直接访问已知 IP，以区分解析问题和路由、端口问题。

1.11 DNS 使用的传输协议

传统 DNS：

- 常用 UDP 53 发送短查询；
- 响应过大或区域传送时使用 TCP 53；
- 客户端在需要重试或避免截断时也可使用 TCP。

加密 DNS：

方式	常见端口	作用
DNS over TLS	TCP 853	加密 DNS 消息
DNS over HTTPS	TCP 443	把 DNS 放入 HTTPS 传输
DNSCrypt	自定义	加密和认证 DNS 通信

加密只保护传输，并选择了一个可信解析器并不表示解析结果一定真实。DNSSEC 是另一套对记录来源进行验证的机制。

1.12 输入 URL 后的实际路径

```
1. 浏览器解析 URL
2. 查询浏览器 DNS 缓存
3. 查询系统缓存和递归解析器
4. 获得 A/AAAA 地址
5. 建立 TCP 或 QUIC 连接
6. 进行 TLS 握手
7. 发送 HTTP 请求
8. 接收并渲染响应
```

DNS 只是开始，后续仍可能遇到路由、端口、TLS、代理和应用程序错误。

2. 代理服务器

2.1 代理的共同结构

代理位于两个通信端点之间：

```
请求方 -> 代理 -> 目标
请求方 <- 代理 <- 目标
```

代理可能建立两条连接，也可能在应用协议层解析请求后再发起一条新连接。

关键问题：

1. 客户端是否明确配置代理？
2. 代理收到请求后是否重新构造请求？
3. 目标服务器看到的是客户端、代理还是其他出口地址？
4. 连接是透明转发还是代理自己生成的新连接？

2.2 正向代理代表客户端

正向代理位于客户端之前：

```
客户端
-> 正向代理
    -> 目标服务器
```

客户端知道自己正在使用代理，通常需要配置：

- 代理地址；
- 端口；
- HTTP/SOCKS 等协议；
- 访问规则；
- 认证凭据。

目标服务器看到的是代理的来源地址，而不是客户端真实地址。

可以将正向代理比作“代购”：商家看到的是代购者，但实际需求来自客户端。

2.3 正向代理的用途

用途	说明
访问控制	企业拒绝或允许特定网站
内容过滤	过滤请求和响应
缓存	保存常用资源，减少回源
匿名隔离	隐藏客户端网络地址
审计	记录代理范围内的请求元数据
统一出口	所有员工经过同一个受控边界
API 调试	开发时观察 HTTP 流量

加速器、企业出口代理和 SOCKS 代理都属于不同形式的正向代理或隧道。

2.4 HTTP 代理

访问明文 HTTP 时，客户端可以向代理发送明确请求：

```
GET http://example.com/index.html HTTP/1.1
Host: example.com
```

代理可以：

- 按 URL 选择路由；

- 修改或删除请求头；
- 查询缓存；
- 执行访问控制；
- 把响应返回客户端。

2.5 HTTPS CONNECT 隧道

HTTPS 正向代理通常先建立隧道：

```
客户端 -> CONNECT example.com:443
代理 -> 与 example.com 建立连接
客户端 <-> TLS 加密数据经代理转发
```

在 CONNECT 模式下，普通代理只能看到目标主机、端口和后续 TLS 元数据，不能直接读取加密正文。

如果代理能读取 HTTPS 内容，通常意味着它通过受信任证书执行中间人解密。这是另一层安全架构，不应与普通透明隧道混为一谈。

2.6 反向代理代表服务器

反向代理位于服务端之前：

```
客户端
-> 反向代理
    -> 后端服务器 1
    -> 后端服务器 2
```

客户端看不到真实后端。它认为自己访问的就是网站本身，而反向代理负责：

- 负载均衡；
- TLS 终止；
- 隐藏后端地址；
- 缓存；
- URL 和请求头重写；
- ACL、限流和鉴权；
- 动静分离；

- 健康检查。

可以把反向代理比作“提前囤货的卖家”：客户端只面对前台，后台具体由谁处理并不暴露。

2.7 CDN 与反向代理

CDN 可以理解为部署在多个地理位置、具有调度和大规模缓存能力的反向代理体系。

```
用户 -> 就近 CDN 节点 -> 回源到业务服务器
```

静态图片、CSS、JS 通常适合从 CDN 缓存返回；需要动态计算或用户会话的请求则可能直接回源。

CDN 不是 Nginx 反向代理的同义词，而是包括调度、缓存、回源、安全和边缘计算的一组系统能力。

2.8 代理不全是第七层

代理的工作层次取决于协议和实现：

类型	特点
HTTP 正向代理	理解 HTTP，可修改请求和响应
HTTP 反向代理	理解 HTTP，可路由和缓存
SOCKS 代理	提供通用 TCP/UDP 转发，通常不解析 HTTP 语义
TCP 负载均衡器	四层连接转发，不解析应用消息
透明代理	不需要客户端显式配置，通过网络重定向拦截

因此不能说“代理必然在应用层”。准确的说法是代理参与连接或请求的中间转发，而是否理解应用协议取决于具体实现。

3. NAT 与代理的区别

NAT 和代理看起来都像“中转”，但控制点不同。

维度	NAT/NAPT	正向代理	反向代理
主要目的	地址复用、内外网互通	代表客户端访问目标	代表服务器接待客户端
主要修改	IP，NAPT 还改端口	客户端向代理请求，代理重新访问目标	代理重新访问后端
对端感知	通常透明	客户端通常明确配置	客户端以为访问的是网站
状态	地址/端口映射	连接、认证、缓存	连接、路由、缓存、TLS
能否解析内容	通常不解析载荷	取决于 HTTP/TLS 模式	取决于协议和 TLS 终止

可以把三者概括为：

```
NAT：改变网络身份
正向代理：帮客户端去找服务器
反向代理：帮服务器接待客户端
```

4. 内网穿透

4.1 NAT 为什么阻挡入站

家庭用户访问公网时，NAPT 创建状态：

```
内网 192.168.1.20:50000
公网 122.77.241.4:60001
远端 example.com:443
```

外部如果直接访问 `122.77.241.4:60001`，NAT 至少必须能够把目的地址和端口映射回内部主机。没有静态映射时，这条入站流量没有对应状态。

可能建立入站的方案：

- 手动端口映射；
- UPnP/NAT-PMP/PCP；
- 运营商或企业防火墙配置；
- 公网服务器中继；

- 支持自动发现的 P2P 穿透。

4.2 NAT 打洞

双方都在 NAT 后面时，可以借助公网协调服务器：

```
客户端 A -> STUN/协调服务器 <- 客户端 B
协调服务器交换双方公网映射
A 与 B 尝试直接互发报文
```

若 NAT 允许已有映射接收外部报文，双方可能直接建立连接。

成功取决于：

- NAT 类型；
- 端口映射是否稳定；
- 是否有保活；
- 是否使用 UDP、TCP 或 QUIC；
- 防火墙策略；
- 对端地址和端口如何被发现。

对称 NAT、CGNAT和严格过滤都会降低成功率。失败后通常转向公网中继。

4.3 打洞与中继

方案	数据路径	特点
NAT 打洞	A 与 B 尽量直连	带宽和延迟更接近直连
公网中继	A -> 服务器 -> B	兼容性强，但依赖服务器带宽
反向隧道	内网主动连公网，外部访问公网入口	不要求公网先访问内网

不能把三者统称为“内网穿透”后就忽略底层差异：它们的外在效果相似，数据路径和 NAT 要求并不同。

5. frp 反向隧道

5.1 frps 与 frpc

frp 包含两个进程：

程序	位置	作用
`frps`	公网服务器	接受隧道连接并暴露公网端口
`frpc`	内网设备	主动连接公网，登记本地服务

整体路径：

```
外部用户
-> 公网服务器:remotePort
-> frps
-> frpc 主动维持的隧道
-> 内网服务
```

frpc 首先向公网服务器发起连接，所以通信方向是“内网主动出站”。这自然绕开了公网无法未经映射直接访问内网的问题。

5.2 为什么 frp 不是 NAT 打洞

frp 不尝试让外部用户与内网服务直接建立 P2P 连接。所有数据通常经过公网 frps 中继。

```
打洞：A <-> B
frp：外部用户 <-> frps <-> frpc <-> 内网服务
```

它利用了内网可主动访问公网的规则，而不是让公网主动穿透 NAT。

5.3 服务端配置

最小配置主要是绑定端口：

```
bindPort = 8888
```

生产环境至少应增加认证：

```
bindPort = 8888

auth.method = "token"
auth.token = "CHANGE_ME"
```

如果需要 TLS，可以进一步配置证书和受信任 CA。实际部署还应限制管理端口和公网防火墙规则。

5.4 内网客户端配置

以 SSH 服务为例：

```
serverAddr = "x.x.x.x"
serverPort = 8888

auth.method = "token"
auth.token = "CHANGE_ME"

[[proxies]]
name = "ssh-service"
type = "tcp"
localIP = "127.0.0.1"
localPort = 22
remotePort = 8081
```

字段含义：

字段	含义
`serverAddr`	公网 frps 地址
`serverPort`	frps 监听端口
`localIP`	内网服务地址
`localPort`	内网服务实际端口
`remotePort`	公网服务器暴露端口

外部访问：

```
ssh -p 8081 user@public-server.example
```

执行过程：

1. 内网 frpc 连接 `serverAddr:serverPort`；
2. frpc 告诉 frps 要暴露 `remotePort=8081`；
3. 外部用户访问公网服务器 8081；
4. frps 把流量通过隧道送到 frpc；
5. frpc 连接内网 `127.0.0.1:22`；
6. SSH 响应沿反方向返回。

5.5 暴露 Nginx 服务

如果内网运行 Nginx，可以单独配置一个 HTTP 代理：

```
[[proxies]]
name = "web-service"
type = "tcp"
localIP = "127.0.0.1"
localPort = 80
remotePort = 8082
```

外部通过：

```
http://public-server.example:8082/
```

生产上通常由公网 Nginx/Caddy 监听 80/443，再将域名流量反向代理到 `127.0.0.1:8082`，从而避免用户直接记住非标准端口，并在公网入口统一完成 TLS、访问控制和限流。

5.6 启动与日志

中演示后台运行：

```
nohup ./frps -c ./frps.toml >/dev/null 2>&1 &
nohup ./frpc -c ./frpc.toml >/dev/null 2>&1 &
```

`>/dev/null 2>&1 &` 的含义：

1. `>/dev/null` 丢弃标准输出；
2. `2>&1` 把标准错误重定向到标准输出；
3. `&` 让进程进入后台。

但这会丢弃日志，不适合生产排错。正式环境更适合 systemd：

```
[Unit]
Description=frps
After=network-online.target

[Service]
ExecStart=/usr/local/bin/frps -c /etc/frp/frps.toml
Restart=on-failure
User=frp

[Install]
WantedBy=multi-user.target
```

客户端使用结构相同的 service 文件，只替换可执行文件、配置路径和运行用户。

6. DNS、代理与内网穿透如何串联

用户访问内网发布的服务时，常见路径可能是：

```
1. 浏览器查询 service.example.com
2. DNS 返回公网服务器地址
3. 浏览器连接公网服务器 443
4. 公网 Nginx 终止 TLS
5. Nginx 把请求转给 frps 暴露端口
6. frps 通过隧道交给 frpc
7. frpc 访问内网应用
8. 响应沿反向路径返回
```

在这个场景中：

- DNS 负责找到公网入口；
- 反向代理负责域名、TLS 和路由；
- frp 负责跨 NAT 的隧道；

- 内网应用只需要监听本机或私网地址；
- 公网不再需要直接路由到家庭私有 IP。

6.1 为什么必须分清各层责任

如果服务无法访问，不同层的问题表现相似，但排查方法完全不同：

排查位置	要确认什么
DNS	域名是否解析到公网入口
公网入口	80/443 是否监听、证书是否正常
反向代理	Host、路径和上游端口是否匹配
frps/frpc	隧道是否在线、remotePort 是否冲突
内网服务	应用是否监听正确地址和端口
防火墙	公网入口、内网出口和回程是否放行

只看“网站打不开”无法判断问题在哪一层。

7. 安全边界

7.1 DNS 安全

- 使用可信解析器；
- 配合 DNSSEC 验证关键域名；
- 防止本地 hosts 被恶意篡改；
- 对重要服务设置合理 TTL；
- 监控异常域名和被污染缓存。

7.2 代理安全

- 正向代理可能看到请求元数据；
- TLS 中间人解密会扩大隐私和证书管理风险；
- 反向代理要限制请求大小、方法、路径和上传；
- 代理本身需要高可用和日志审计；
- 不应把管理后台或数据库无鉴权暴露在代理后方。

7.3 内网穿透安全

frp 把内网服务带到公网，至少需要：

1. 使用强 token 和最小权限账户；
2. 只映射必要端口；
3. 禁止直接暴露数据库、Redis 和内部管理接口；
4. 在公网入口启用 TLS、访问白名单和限流；
5. SSH 使用密钥认证并考虑来源限制；
6. 配置文件不提交真实密钥；
7. 监控异常连接和失败登录；
8. 定期更新 frp 和系统组件。

“能穿透”不等于“可以公开”。穿透解决的是可达性，安全仍需单独设计。

8. 系列总回顾

七篇文章对应一次完整网络访问：

```
1. IP 首部和地址划分决定目标网络
2. 私有地址通过 NAT/NAPT 进入公网
3. 路由表逐跳选择下一站
4. 以太网帧在当前链路传输
5. ARP 把下一跳 IP 解析成 MAC
6. MTU 限制报文大小，分片提供 IPv4 兼容路径
7. DNS 找到服务入口，代理和隧道连接不同网络
```

继续向下，应用协议还需要处理连接、请求、状态和业务语义；继续向上，CDN、负载均衡和网关在现代系统中又会叠加更多层次。但无论系统多复杂，基本问题始终是：

```
目标是谁？
怎样找到它？
当前链路怎样交付？
报文太大怎么办？
对方是否可达？
失败时谁报告原因？
```

9. 本篇小结

`DNS` 把域名解析成地址和记录
正向代理代表客户端
反向代理代表服务器
`NAT` 透明改写地址和端口
`frp` 通过公网中继建立反向隧道
`NAT` 打洞尝试点对点直连
穿透解决可达性，不自动解决安全